

# DA TEORIA À PRÁTICA NO DESENVOLVIMENTO DE SISTEMAS

A Fundação da Excelência Pedagógica e Técnica  
na **Modelagem de Dados**





A construção de sistemas de software de alto desempenho e confiabilidade não se inicia

com a escrita de linhas de código em linguagens como Java, Python ou C#, mas sim com a estruturação rigorosa e inteligente da informação que essas linguagens irão manipular. No contexto do Curso Técnico em Desenvolvimento de Sistemas do SENAI MG, a modelagem de dados é compreendida como a arquitetura fundamental de qualquer solução tecnológica.

A Metodologia SENAI de Educação Profissional (MSEP) posiciona o estudante como o protagonista desse processo, exigindo que o futuro técnico desenvolva não apenas a habilidade de executar comandos, mas a capacidade de planejar e tomar decisões autônomas baseadas em critérios técnicos sólidos e nas demandas do mundo produtivo.

A importância de uma boa modelagem reflete diretamente no sucesso ou no fracasso de um software. Um modelo de dados bem projetado atua como o alicerce de um edifício: se ele for frágil ou mal planejado, toda a estrutura superior — incluindo a interface do usuário e as regras de negócio — estará sujeita a falhas catastróficas, problemas de performance e inconsistências que podem levar à perda de dados críticos.

Em um mercado cada vez mais orientado a dados, como o ecossistema tecnológico de Minas Gerais, a precisão na modelagem garante que a aplicação seja escalável e de fácil manutenção, reduzindo o que o setor chama de "dívida técnica".<sup>5</sup>

A modelagem de dados deve ser vista como um processo de tradução. O desenvolvedor atua como um intérprete que recebe os requisitos de negócio, muitas vezes abstratos e informais, e os transforma em esquemas técnicos rigorosos que um Sistema Gerenciador de Banco de Dados (SGBD) possa processar eficientemente.

Este guia técnico detalha as etapas dessa tradução, focando na transição entre a abstração lógica e a implementação física, preparando o aluno para os desafios reais encontrados em empresas de Belo Horizonte, Contagem e outras regiões industriais do estado.

## **O Percurso da Abstração: Da Concepção à Implementação Real**

O design de um banco de dados relacional não é um evento único, mas um ciclo de refinamento contínuo dividido em três estágios principais: conceitual, lógico e físico.<sup>4</sup> Cada estágio atende a um público diferente e possui um nível de detalhamento técnico crescente.

Enquanto o modelo conceitual utiliza o Diagrama Entidade-Relacionamento (DER) para capturar a visão global do negócio sem se preocupar com detalhes de implementação, os modelos lógico e físico entram na esfera técnica do desenvolvimento de sistemas.

### **O Modelo Lógico: A Estrutura Independente de Tecnologia**

O modelo lógico de dados funciona como uma ponte entre a visão de negócio e a implementação técnica. Ele é uma versão refinada do modelo conceitual, onde as entidades e relacionamentos são traduzidos para uma estrutura que já contempla nomes de atributos, chaves primárias e relacionamentos, mas que ainda mantém total independência do software de banco de dados que será utilizado.

Nesta fase, o desenvolvedor ou arquiteto de dados define como os objetos de dados operam e transacionam de forma que as partes interessadas da empresa possam validar as regras. Por exemplo, em um sistema de bilheteria, o modelo lógico identificaria a entidade "Pessoa" com atributos como nome\_completo e data\_nascimento, definindo qual desses campos

identificará cada registro de forma única, mas sem especificar se o campo será um VARCHAR ou um TEXT.

O foco aqui é a lógica de armazenamento e a integridade das relações, garantindo que todos os requisitos funcionais levantados na fase de análise de sistemas foram devidamente mapeados para estruturas de dados.

## **O Modelo Físico: A Realidade Técnica do SGBD**

O modelo físico é o destino final do projeto de dados. Ele refina o modelo lógico para uma implementação concreta em uma tecnologia específica, como MySQL, SQL Server, PostgreSQL ou Oracle.

Neste nível de detalhamento, as decisões deixam de ser puramente lógicas e passam a considerar aspectos de hardware, performance de disco, otimização de consultas e as restrições gramaticais da linguagem SQL de cada plataforma.

No modelo físico, o desenvolvedor define tipos de dados precisos (como INT, DECIMAL(10,2), TIMESTAMP), cria índices para acelerar a recuperação de informações, estabelece procedimentos armazenados (stored procedures) e define gatilhos (triggers) que automatizam tarefas complexas.

É o esquema que será efetivamente executado no servidor para criar as tabelas e gerenciar o armazenamento físico no disco.

| <b>Critério de Comparação</b>    | <b>Modelo Lógico de Dados</b>               | <b>Modelo Físico de Dados</b>                   |
|----------------------------------|---------------------------------------------|-------------------------------------------------|
| <b>Dependência de Plataforma</b> | Totalmente independente de SGBD.            | Específico para uma tecnologia (ex: MySQL).     |
| <b>Componentes Principais</b>    | Entidades, Atributos e Relacionamentos.     | Tabelas, Colunas, Tipos de Dados e Índices.     |
| <b>Público-Alvo</b>              | Analistas de Negócio e Arquitetos de Dados. | Desenvolvedores e Administradores (DBAs).       |
| <b>Objetivo Primário</b>         | Representar regras e fluxos de negócio.     | Otimizar armazenamento e performance.           |
| <b>Atributos Técnicos</b>        | Não inclui triggers ou procedures.          | Inclui scripts SQL, triggers e índices físicos. |

**Fonte:** Baseado em documentação técnica da AWS e metodologias de design de dados.

## Mecanismos de Integridade e o Poder das Constraints

No desenvolvimento de sistemas profissionais, a confiança nos dados é o ativo mais valioso de uma organização. Para garantir que as informações inseridas em um banco de dados sejam precisas e consistentes, utilizamos as **Constraints** (Restrições).

Elas são regras impostas às colunas das tabelas que o SGBD deve validar antes de permitir qualquer alteração nos dados. Sem o uso correto de restrições, o banco de dados correria o risco de armazenar informações contraditórias, como um produto com preço negativo ou um pedido vinculado a um cliente inexistente.

### Primary Key (PK): A Identidade Inviolável

A Chave Primária é o elemento que garante que cada linha de uma tabela seja única. Em um curso técnico do SENAI, ensinamos que uma tabela sem PK é como uma sala de aula onde todos os alunos têm o mesmo nome e não possuem documentos de identidade: é impossível distinguir um do outro com precisão.

Tecnicamente, a PK impõe duas condições automáticas: a exclusividade (não pode haver dois valores iguais) e a não-nulidade (o campo não pode ficar vazio). Em sistemas de e-commerce, por exemplo, o `id_produto` é a PK que diferencia um smartphone de uma capa de proteção, mesmo que ambos tenham descrições semelhantes. A escolha da PK é estratégica, pois o banco de dados cria um índice sobre essa coluna para tornar as buscas extremamente rápidas.

## Foreign Key (FK): A Trama dos Relacionamentos

A Chave Estrangeira é o mecanismo que "costura" as tabelas entre si, estabelecendo a integridade referencial. Uma FK em uma tabela aponta para a PK de outra, garantindo que o relacionamento entre os dados seja legítimo. Se tentarmos apagar um cliente que possui pedidos ativos, o SGBD impedirá a ação, protegendo o sistema de gerar "dados órfãos".

Imagine uma biblioteca: a tabela Emprestimos possui uma FK chamada id\_livro que referencia a tabela Livros. Essa restrição assegura que nenhum empréstimo seja registrado para um livro que não conste oficialmente no acervo da biblioteca.

## Unique: A Garantia de Exclusividade Adicional

A restrição UNIQUE impede a duplicidade de valores em colunas que não são necessariamente a chave primária.<sup>11</sup> Diferente da PK, uma tabela pode ter várias colunas com a restrição UNIQUE, e elas permitem, em geral, um valor nulo por coluna.

Um exemplo clássico no mercado brasileiro é o campo E-mail em cadastros de usuários. Embora o sistema use um id\_usuario numérico como PK para performance, o e-mail deve ser único para evitar que duas pessoas tentem se cadastrar com a mesma conta.

## Not Null e Check: A Validação de Negócio no Banco

A restrição NOT NULL é a forma mais simples de garantir que informações obrigatórias sejam fornecidas. Em um sistema de RH, o nome do funcionário e a data de admissão jamais devem ser nulos. Já a restrição CHECK é uma das ferramentas mais flexíveis, pois permite criar expressões lógicas para validar os dados.



Por exemplo, em um sistema financeiro, podemos aplicar CHECK (valor\_transacao > 0) para impedir que transações de valor nulo ou negativo sejam processadas por erro de programação no front-end

| Constraint              | Função Principal                                     | Exemplo de Aplicação Real           |
|-------------------------|------------------------------------------------------|-------------------------------------|
| <b>Primary Key (PK)</b> | Identificar cada registro de forma única e não nula. | id_cliente em um sistema de banco.  |
| <b>Foreign Key (FK)</b> | Vincular registros entre tabelas diferentes.         | id_categoria na tabela de produtos. |
| <b>Unique</b>           | Garantir que não existam valores duplicados.         | numero_cpf ou endereco_email.       |
| <b>Not Null</b>         | Impedir que um campo fique vazio.                    | data_nascimento em fichas médicas.  |
| <b>Check</b>            | Validar valores baseando-se em uma lógica.           | idade >= 18 para sites restritos.   |

Fonte: Síntese de conceitos de integridade em SQL Server e MySQL.

# Melhores Práticas de Design e Engenharia de Banco de Dados

O design de banco de dados não é apenas uma tarefa técnica, mas uma forma de arte funcional que exige disciplina e visão de longo prazo. Como instrutor, enfatizo que a "alma" do sistema reside em seus dados, e a forma como projetamos as tabelas determinará a longevidade da aplicação.

No mercado de TI, especialmente em polos competitivos como Minas Gerais, a adesão a padrões de nomenclatura e boas práticas de estruturação é o que diferencia um desenvolvedor júnior de um profissional sênior.

## Convenções de Nomenclatura: Clareza e Consistência

A nomenclatura de tabelas e colunas deve ser intuitiva e consistente. Embora não exista uma regra única universal, a prática comum no desenvolvimento de software sugere o uso de nomes no singular para tabelas, pois cada linha representa uma entidade única (ex: Produto em vez de Produtos). Além disso, deve-se evitar a redundância de incluir o nome da tabela em cada coluna (ex: em vez de cliente\_nome na tabela Cliente, use apenas nome).

Em grandes projetos, a internacionalização é uma tendência forte. O uso de nomes em inglês (como orders, customers) é padrão em muitas multinacionais em Belo Horizonte para facilitar a colaboração com equipes globais, mas em projetos locais o português é aceitável, desde que sem acentos ou caracteres especiais que possam causar erros de codificação no SGBD.

## Estruturação de Relacionamentos N:N

Um erro comum entre alunos iniciantes é tentar forçar relacionamentos de Muitos-para-Muitos (N:N) dentro de uma única tabela usando campos multivalorados ou listas de strings. A boa prática de design exige a criação de uma **Tabela Associativa** (ou tabela de junção).

Se um aluno pode se matricular em várias disciplinas e uma disciplina tem vários alunos, criamos a tabela Matricula, que conterà chaves estrangeiras para Aluno e Disciplina. Essa abordagem mantém a atomicidade dos dados e facilita a geração de relatórios e a manutenção da integridade referencial.

## O Conceito de Dependência Funcional e o Caminho para a Normalização

Para que um banco de dados seja eficiente, ele deve passar pelo processo de normalização, que visa eliminar redundâncias e anomalias de atualização. O alicerce desse processo é a compreensão da **Dependência Funcional**.

Em termos simplificados para o início do curso, dizemos que uma coluna  $Y$  depende funcionalmente de uma coluna  $X$  se, ao conhecermos o valor de  $X$ , conseguimos determinar com 100% de certeza o valor de  $Y$ .

Nessa relação, chamamos  $X$  de **Determinante** e  $Y$  de **Dependente**. Em uma tabela de veículos, o numero\_chassi determina o modelo e o ano. Se você mudar o chassi, o modelo e o ano podem mudar; mas se você souber o chassi, o modelo é fixo.

A dependência funcional é a ferramenta que o desenvolvedor usa para decidir se um dado deve permanecer em uma tabela ou se deve ser movido para uma nova, evitando que a mesma informação seja repetida desnecessariamente.

## Tipos de Dependência e suas Implicações

Existem variações importantes que o técnico deve identificar para aplicar as formas normais corretamente:

1. **Dependência Total:** Quando um campo depende de toda a chave primária. Em uma tabela de Itens\_Pedido onde a PK é composta por (id\_pedido, id\_produto), a quantidade vendida depende de ambos os campos juntos.
2. **Dependência Parcial:** Ocorre quando um campo depende de apenas uma parte de uma chave composta. Isso indica que a tabela precisa ser dividida para atingir a Segunda Forma Normal (2FN).
3. **Dependência Transitiva:** Ocorre quando um campo depende de outro campo que não é a chave primária. Se na tabela de Livro o nome\_editora depende do id\_editora, e este depende do id\_livro, temos uma dependência transitiva que deve ser resolvida para alcançar a Terceira Forma Normal (3FN).

Compreender essas dependências permite que o desenvolvedor evite as famosas "anomalias de inserção", onde você não consegue cadastrar um dado porque ele depende de outro que ainda não existe, ou "anomalias de remoção", onde ao apagar um registro você perde acidentalmente informações que deveriam ser preservadas em outra parte do sistema.



# **O Mercado de TI em Minas Gerais: Realidades e Tendências para 2026**

Como instrutor do SENAI MG, é meu dever preparar vocês para a realidade das empresas em nossa região. Minas Gerais possui um dos mercados de tecnologia mais vibrantes do país, com destaque para o setor financeiro, construção civil, logística e saúde. Para os próximos anos, algumas tendências fundamentais moldarão o trabalho com bancos de dados.

## **Exigências das Empresas em Belo Horizonte e Região**

Empresas como a MRV, com sede em Belo Horizonte, e diversas consultorias de TI buscam profissionais que não apenas saibam modelar, mas que entendam de análise de dados e dashboards em ferramentas como Power BI.

Vagas de nível júnior em instituições como a UFMG ou em empresas de logística em Contagem frequentemente exigem conhecimento sólido em T-SQL (SQL Server) e lógica de programação aliada a bancos relacionais.

O mercado valoriza o "trabalhador integral", aquele que possui as competências técnicas (hard skills) de modelagem, mas também as capacidades socioemocionais (soft skills) de comunicação e trabalho em equipe.

Em 2025, o domínio de ferramentas de nuvem como AWS, Azure e Google Cloud será um diferencial indispensável para quem deseja atuar em projetos de grande escala.

## Inovações e o Impacto da IA na Gestão de Dados

A Inteligência Artificial e o Machine Learning estão transformando a forma como os dados são processados. Profissionais que souberem utilizar a IA para otimizar modelos de dados e automatizar tarefas repetitivas estarão na vanguarda do mercado.

Além disso, a cibersegurança e a privacidade de dados (em conformidade com a LGPD) tornaram-se prioridades críticas. O design de um banco de dados hoje deve prever a proteção de informações sensíveis desde o primeiro diagrama lógico.

### Dica do Instrutor: O Diferencial do Técnico SENAI

"No dia a dia de um desenvolvedor, você raramente terá todo o tempo do mundo para modelar. A pressão por entregas rápidas é real. No entanto, nunca sacrifique a integridade do seu banco por velocidade.

Um relacionamento mal feito hoje será o bug de performance impossível de resolver daqui a seis meses.

Em Minas, as empresas buscam o 'time sangue verde': profissionais engajados, protagonistas de sua aprendizagem e que entregam soluções com qualidade industrial. Pratiquem a normalização até que ela se torne intuitiva, e lembrem-se: o dado é o maior patrimônio de uma empresa. Trate-o com o respeito técnico que ele merece."

## Exercícios de Fixação: Estudos de Caso Práticos

Para consolidar os conceitos de modelagem lógica, física e integridade de dados, resolva os seguintes estudos de caso baseados em cenários reais de mercado.

### Estudo de Caso 1: Modernização da Biblioteca Municipal

**Cenário:** Uma biblioteca deseja substituir suas planilhas de papel por um sistema de banco de dados. Eles precisam armazenar informações sobre livros (título, autor, ISBN, ano), usuários (nome, CPF, endereço, e-mail) e os registros de empréstimos.

**Tarefa:**

1. Crie um Modelo Lógico identificando as entidades e seus atributos.
2. Defina quais seriam as Chaves Primárias e Estrangeiras necessárias para garantir que um livro não possa ser emprestado a um usuário que não esteja cadastrado.
3. Aplique uma restrição UNIQUE em um campo que você considere crítico para evitar duplicidade de cadastros.

### Estudo de Caso 2: Sistema de Controle de Estoque para Autopeças

**Cenário:** Uma loja de autopeças em Contagem está enfrentando problemas com dados inconsistentes. Às vezes, o sistema aceita quantidades negativas no estoque ou preços zerados.

**Tarefa:**

1. No nível de Modelo Físico, descreva como você utilizaria a restrição CHECK para impedir que a quantidade em estoque seja menor que zero.
2. Justifique o uso de NOT NULL para o campo nome\_peca e id\_fabricante.
3. Explique como a Foreign Key entre as tabelas Pecas e Fabricantes ajuda a manter a organização da loja.

## **Estudo de Caso 3: Plataforma de Cursos Online (EdTech)**

**Cenário:** Você está projetando o banco de dados para uma nova plataforma de ensino. Um aluno pode se matricular em vários cursos, e cada curso pode ter milhares de alunos matriculados.

### **Tarefa:**

1. Como você resolveria esse relacionamento Muitos-para-Muitos (N:N)? Desenhe a estrutura das tabelas necessárias no modelo lógico.
2. Identifique uma possível Dependência Funcional Transitiva nesse cenário (por exemplo, se colocássemos o nome do professor e sua especialidade diretamente na tabela de cursos) e explique como resolvê-la através da normalização.
3. Quais índices você criaria no modelo físico para garantir que a busca por cursos por categoria seja rápida para o usuário final?

***Este Guia de Estudo Técnico é um material de apoio para a Unidade Curricular de Banco de Dados. Utilize-o como consulta constante em suas atividades laboratoriais e projetos integradores. O sucesso na modelagem é o alicerce de sua carreira como Desenvolvedor de Sistemas.***